Glyph Foundry

A Deterministic Crystallization Core for Evolutionary Glyph-Languages

Unified with the Deterministic Information Crystallization Framework

Specification v2.1 — Normative

Sebastian Klemm

March 2026

Abstract

This document specifies a standalone, production-oriented core system for *glyph-languages*: programming surfaces whose tokens, operators, and control symbols are written in arbitrary Unicode scripts, while program semantics are represented in a single canonical intermediate representation (IR) and processed under deterministic, replayable policies.

Version 2.1 supersedes v2.0 by unifying the Glyph Foundry's engineering-grade pipeline with the formal algebraic structure of *Deterministic Information Crystallization* (DIC). The result is a system that not only lowers glyph-language sources to canonical IR and emits verifiable bundles, but also embeds each IR graph into a five-dimensional measurement space $\mathbb{H}_5 = \mathbb{Q}_{16}^5$, tracks convergence via Temporal Information Certificates, and produces immutable, cryptographically certified knowledge units called *Crystals* when the crystallization condition is satisfied.

The specification defines: (i) deterministic fixed-point arithmetic on $\mathbb{Q}_{16}$; (ii) the embedding space $\mathbb{H}_5$ with axis functions, fingerprints, and kernel distance; (iii) convergence tracking and the crystallization protocol; (iv) the Morphogenetic Evidence Framework (MEF) as a hash-chained evidence ledger; (v) a constraint lattice and gate system; (vi) normative JSON schemas for all wire-format objects; (vii) a concrete CLI contract; (viii) golden test vectors; (ix) a threat and failure model; and (x) a domain-agnostic adapter boundary.

All requirements use RFC 2119 keywords.

Contents

1 Scope **5**
1.1 In-Scope 5
1.2 Out-of-Scope 5

2 Conformance **5**

3 Terminology **6**

4 Foundations **6**
4.1 Deterministic Arithmetic ($\mathbb{Q}_{16}$) . . . 6
4.2 The Canonical Digest . . . 7

5 The Information Graph **7**

6 The Embedding Space $\mathbb{H}_5$ **8**
6.1 Definition . . . 8
6.2 Fingerprint . . . 8
6.3 Distance and Kernel . . . 8

7 Observable Alphabet and Information Genome **9**

8 Determinism Contract **9**

9 Run Descriptor (RD) **9**
9.1 Normative JSON Schema . . . 9
9.2 RD Computation Rules . . . 11

10 System Model **11**
10.1 Core State . . . 11
10.2 Execution Model: The Crystallization Pipeline . . . 11

11 Unicode Policy **12**
11.1 Normalisation . . . 12
11.2 Forbidden and Restricted Characters . . . 12
11.3 Confusable Policy . . . 12
11.4 Unicode-Specific Threats . . . 12

12 Canonical IR (Wire Format) **13**
12.1 Single Truth Requirement . . . 13
12.2 IR Node Schema v1.0 . . . 13
12.3 IR Edge Schema v1.0 . . . 13
12.4 IR Document Schema v1.0 . . . 13
12.5 Canonical Serialisation Invariants . . . 14

13 Deterministic Ordering Rules **14**
13.1 Canonical Node Ordering ($\prec_V$) . . . 14
13.2 Canonical Edge Ordering ($\prec_E$) . . . 14
13.3 Stable Node ID Allocation . . . 14

14 Convergence Tracking **15**
14.1 Temporal Information Certificate (TIC) . . . 15
14.2 Condensation Predicate . . . 15
14.3 Crystallization Condition . . . 15
14.4 Contractivity and Convergence . . . 15

15 Crystal: The Immutable Knowledge Unit **16**
15.1 Crystal JSON Schema . . . 16

16 Morphogenetic Evidence Framework (MEF) 17
16.1 MEF Block JSON Schema . . . 18

17 Gate System 18
17.1 Obligation Registry Schema . . . 18

18 Contractive Fixpoint Iteration 19

19 Registries 19
19.1 Operator Registry . . . 19
19.2 Macro Registry . . . 20
19.3 Observable Registry . . . 20
19.4 Registry Hash Binding . . . 20

20 Macro Expansion 20

21 Reference Surface Language: Sanskroot-Devanagari 21
21.1 Minimum Vocabulary (Normative) . . . 21

22 Bundle and Evidence Format 21
22.1 Bundle Layout (Normative) . . . 21
22.2 Manifest Schema . . . 21
22.3 Verification . . . 22

23 CLI Interface 22
23.1 Binary Name . . . 22
23.2 Commands and Exit Codes . . . 22
23.2.1 `glyph init-rd` . . . 22
23.2.2 `glyph run` . . . 22
23.2.3 `glyph verify` . . . 23
23.2.4 `glyph dump-ir` . . . 23
23.2.5 `glyph embed` . . . 23

24 Golden Test Vectors 23
24.1 Test Vector 1: Minimal Print . . . 23
24.2 Test Vector 2: Conditional Branch . . . 24
24.3 Test Vector 3: Multi-Function with Return . . . 24
24.4 Golden Path Determinism Test . . . 24

25 Domain Adaptation and Integration Boundary 24
25.1 Domain Adapter . . . 24
25.2 Example Domain Adapters . . . 25
25.3 Integration Boundary . . . 25

26 Composition: Recursive Crystallization 25

27 Formal Properties 26

28 Threat and Failure Model 27

29 Repository Blueprint **28**

30 Evolution Staircase **28**
30.1 Stable Core (MUST remain invariant) 28
30.2 Mutable Surface (MAY evolve) 29

31 Definition of Done (Production Readiness) **29**

A Canonical JSON Serialisation Algorithm **29**

B Digest Computation Reference **30**

1 Scope

1.1 In-Scope

(a) Deterministic fixed-point arithmetic on $\mathbb{Q}_{16}$ as the computational substrate.

(b) Canonical Unicode handling for glyph-language sources.

(c) Deterministic lowering from glyph sources to a single canonical IR wire format.

(d) Embedding of IR graphs into the five-dimensional space $\mathbb{H}_5$ via domain-parameterized axis functions.

(e) Convergence tracking via Temporal Information Certificates (TIC).

(f) Crystallization: production of immutable, hash-anchored Crystals upon convergence.

(g) The Morphogenetic Evidence Framework (MEF) as a tamper-evident, append-only hash chain.

(h) A constraint lattice and gate system on $\mathbb{H}_5$.

(i) Versioned, content-addressed registries for operators, macros, observables, profiles, and obligations.

(j) Normative JSON schemas for all wire-format objects.

(k) A concrete CLI contract with commands, flags, exit codes, and filesystem expectations.

(l) Golden test vectors for the reference Sanskroot-Devanagari frontend.

(m) A threat and failure model addressing Unicode pitfalls, replay integrity, and convergence anomalies.

(n) A domain-agnostic adapter boundary for forward-compatible integration.

1.2 Out-of-Scope

(a) Claims about correctness of external domains. All domain intent is expressed only via Domain Adapters.

(b) Full compiler-grade semantic coverage (types, borrow checking, full CFG/SSA) beyond the minimal IR.

(c) Runtime execution semantics (interpreting/compiling the IR to machine behaviour).

(d) Any requirement to depend on a specific existing repository.

2 Conformance

An implementation conforms to this specification if and only if it satisfies every requirement marked **MUST** or **MUST NOT**. Requirements marked **SHOULD** are normative recommendations; deviations **MUST** be documented. Requirements marked **MAY** are optional. The key words are interpreted as described in RFC 2119.

3 Terminology

Term	Definition
$\mathbb{Q}_{16}$	Signed 32-bit fixed-point field: integers interpreted as $q/2^{16}$, range $[-32768, 32767.999984]$, resolution $\approx 1.526 \times 10^{-5}$. No floating-point arithmetic participates in identity-bearing computations.
$\mathbb{H}_5$	Embedding Space: $\mathbb{Q}_{16}^5$. A 5-dimensional normed space of measurement vectors.
Information Graph	A directed, typed, labelled graph $\mathcal{G} = (V, E, \tau_V, \tau_E, \mu)$ serving as the universal intermediate representation.
Glyph-language	A surface language whose tokens **MAY** be Unicode glyphs outside Basic Latin.
IR	Canonical Intermediate Representation; a schema-versioned JSON wire format (Section 12).
RD	Run Descriptor; configuration object binding policies, versions, and seeds (Section 9).
Crystal	An immutable, cryptographically certified knowledge unit produced upon convergence (Section 15).
TIC	Temporal Information Certificate; a sequence of convergence observations (Section 14).
MEF	Morphogenetic Evidence Framework; an append-only hash chain of evidence blocks (Section 16).
Bundle	A filesystem package containing normalised source, IR, evidence, trace, manifest, and artifacts.
Canonical JSON	JSON serialisation: keys sorted lexicographically by UTF-8 bytes, arrays preserve declared order, no whitespace between tokens, no floating-point literals.
Digest	`SHA-256` hash computed over canonical JSON bytes unless otherwise stated. Equivalently: $\mathrm{dig}(o) = \mathrm{SHA256}(\mathrm{JCS}(o))$.
Observable	A deterministic measurement function $\mathcal{O} : \mathcal{G} \to \mathbb{Q}_{16}^k$ extracting a k-dimensional vector from an Information Graph.
Domain Adapter	A tuple $\mathcal{D} = (\mathcal{T}_V, \mathcal{T}_E, \mathcal{O}_1 \ldots \mathcal{O}_m, f_1 \ldots f_5, \pi, \varepsilon, n)$ parameterizing the framework for a specific domain (Section 25).
Registry	Versioned, content-addressed catalogue of operators, macros, observables, or obligations.

4 Foundations

4.1 Deterministic Arithmetic ($\mathbb{Q}_{16}$)

Axiom 4.1 (Deterministic Arithmetic)**.** All numerical computation in the framework is performed in the fixed-point field $\mathbb{Q}_{16}$: signed 32-bit integers interpreted as $q/2^{16}$, with range $[-32768, 32767.999984]$ and resolution $\approx 1.526 \times 10^{-5}$. Division uses banker's rounding (round-half-to-even). No floating-point arithmetic participates in identity-bearing computations.

This axiom eliminates all sources of non-determinism from platform-dependent floating-point

behaviour. Two implementations on different hardware, given the same input, **MUST** produce bit-identical results.

4.2 The Canonical Digest

Definition 4.1 (Canonical Digest)**.** For any serializable object o, the canonical digest is:

$$\mathrm{dig}(o) = \mathrm{SHA256}\big(\mathrm{JCS}(o)\big) \in \{0,1\}^{256}$$

where JCS denotes JSON Canonicalization Scheme (RFC 8785): deterministic JSON serialisation with lexicographically sorted keys and normalised number representation. The output is a 64-character lowercase hexadecimal string. All values quantized to $\mathbb{Q}_{16}$ before canonicalization.

Theorem 4.1 (Digest Properties)**.** *The canonical digest satisfies:*

1. ***Determinism:*** $o_1 = o_2 \implies \mathrm{dig}(o_1) = \mathrm{dig}(o_2)$.
2. ***Collision resistance:*** *inherited from SHA-256.*
3. ***Platform independence:*** *guaranteed by* $\mathbb{Q}_{16}$ *quantization and JCS key ordering.*

5 The Information Graph

Definition 5.1 (Information Graph)**.** An **Information Graph** is a directed graph $\mathcal{G} = (V, E, \tau_V, \tau_E, \mu)$ where:

- V is a finite set of vertices (information units).
- $E \subseteq V \times V$ is a set of directed edges (relations).
- $\tau_V : V \to \mathcal{T}_V$ assigns each vertex a type from a finite type alphabet $\mathcal{T}_V$.
- $\tau_E : E \to \mathcal{T}_E$ assigns each edge a type from a finite type alphabet $\mathcal{T}_E$.
- $\mu : V \cup E \to \mathcal{M}$ assigns metadata (a JSON-serializable value) to each vertex and edge.

The type alphabets $\mathcal{T}_V$ and $\mathcal{T}_E$ are **domain-specific parameters** defined by the Domain Adapter (Section 25). The graph itself carries no domain semantics.

Definition 5.2 (Graph Digest)**.** The digest of an Information Graph $\mathcal{G}$ is:

$$\mathrm{dig}(\mathcal{G}) = \mathrm{SHA256}\big(\mathrm{JCS}(\mathcal{G})\big)$$

where the JCS representation uses sorted vertex/edge lists keyed by vertex ID.

The Information Graph is the universal intermediate representation of the framework. Every domain projects its native objects into this graph structure via the Domain Adapter.

6 The Embedding Space $\mathbb{H}_5$

6.1 Definition

Definition 6.1 (Embedding Space)**.** The **Embedding Space** is a 5-dimensional normed space:

$$\mathbb{H}_5 = \mathbb{Q}_{16}^5$$

A point $\mathbf{e} = (a_1, a_2, a_3, a_4, a_5) \in \mathbb{H}_5$ represents the essential characteristics of an Information Graph along five independent measurement axes. Each $a_i \in \mathbb{Q}_{16}$.

Definition 6.2 (Axis Semantics)**.** The five axes are **domain-parameterized**. Each axis is computed by an **axis function** $f_i : \mathcal{G} \times W_i \to \mathbb{Q}_{16}$ where W_i is a weight vector. The framework requires only:

1. Each f_i is deterministic: $\mathcal{G}_1 = \mathcal{G}_2 \wedge W_1 = W_2 \implies f_i(\mathcal{G}_1, W_1) = f_i(\mathcal{G}_2, W_2)$.
2. Each f_i depends only on the graph structure (vertices, edges, types, metadata) — no external state.
3. The five axes are **semantically independent**: each captures a qualitatively distinct aspect.

For the Glyph Foundry's software domain, the canonical axis mapping is: a_1 = structural coupling, a_2 = functional density, a_3 = topological complexity, a_4 = symmetry, a_5 = entropy.

6.2 Fingerprint

Definition 6.3 (Zero-Float Fingerprint)**.** The fingerprint of an embedding $\mathbf{e} \in \mathbb{H}_5$ is:

$$\text{ZFP}(\mathbf{e}) = \text{SHA256}\big(\text{hex}(a_1) \| \text{":"} \| \text{hex}(a_2) \| \cdots \| \text{hex}(a_5)\big)$$

Since each a_i is a $\mathbb{Q}_{16}$ integer, the fingerprint contains no floating-point representation and is platform-independent.

6.3 Distance and Kernel

Definition 6.4 (Resonance Kernel)**.** The factorized Gaussian RBF kernel on $\mathbb{H}_5$ is:

$$K(\mathbf{e}_x, \mathbf{e}_y; \sigma) = \prod_{i=1}^{5} \exp\left(-\frac{(x_i - y_i)^2}{2\sigma_i^2}\right)$$

with bandwidth parameters $\sigma = (\sigma_1, \ldots, \sigma_5) \in \mathbb{R}_{>0}^5$. The kernel satisfies $K \in [0, 1]$ with $K(\mathbf{e}, \mathbf{e}) = 1$.

Definition 6.5 (Kernel Distance)**.**

$$\delta(\mathbf{e}_x, \mathbf{e}_y; \sigma) = \sqrt{2 - 2\,K(\mathbf{e}_x, \mathbf{e}_y; \sigma)}$$

This is a pseudometric on $\mathbb{H}_5$: $\delta(\mathbf{e}, \mathbf{e}) = 0$, $\delta(\mathbf{e}_x, \mathbf{e}_y) = \delta(\mathbf{e}_y, \mathbf{e}_x)$, and the triangle inequality holds. All comparisons use $\mathbb{Q}_{16}$ integer arithmetic (no epsilon tolerance).

7 Observable Alphabet and Information Genome

Definition 7.1 (Observable)**.** An **Observable** (also called *Resonite*) is a deterministic measurement function:

$$\mathcal{O} : \mathcal{G} \to \mathbb{Q}_{16}^{k}$$

that extracts a k-dimensional measurement vector from an Information Graph. Each observable has a unique identifier id $\in$ String, a fixed arity $k \in \mathbb{N}$, and a deterministic computation rule.

Definition 7.2 (Observable Registry)**.** An **Observable Registry** $\mathcal{R} = \{\mathcal{O}_1, \ldots, \mathcal{O}_m\}$ is a finite, ordered set of observables. The registry is a **domain parameter**: different domains define different sets of observables.

Definition 7.3 (Information Genome)**.** Given an Information Graph $\mathcal{G}$, an Observable Registry $\mathcal{R}$, and a configuration θ, the **Information Genome** is:

$$\Gamma(\mathcal{G}, \mathcal{R}, \theta) = \mathrm{NF}\big(\{(\mathcal{O}_j.\mathrm{id},\ i,\ \mathcal{O}_j(\mathcal{G})_i) \mid \mathcal{O}_j \in \mathcal{R},\ 1 \le i \le k_j\}\big)$$

where NF denotes **Normal Form**: stable lexicographic sorting, deduplication, and identity elimination. The genome identifier is:

$$\texttt{genome_id} = \mathrm{dig}\big(\Gamma(\mathcal{G}, \mathcal{R}, \theta)\big)$$

Two graphs with identical genomes are structurally indistinguishable under the chosen observable set.

8 Determinism Contract

Requirement 8.1 (Total Determinism)**.** The system **MUST** be deterministic with respect to the pair (normalised input bytes, RD).

Requirement 8.2 (Output Identity)**.** Identical normalised input bytes combined with an identical RD **MUST** yield byte-identical: (i) `ir.json`, (ii) `manifest.json` and its digest, (iii) `evidence.json`, (iv) `trace.json`, (v) every file under `artifacts/`, (vi) all $\mathbb{H}_5$ embeddings, fingerprints, TIC records, and Crystal digests.

Requirement 8.3 (No Hidden Stochasticity)**.** Stochastic behaviour **MUST NOT** exist unless the random seed is deterministically derived from a seed value bound in the RD.

Requirement 8.4 (Stable Iteration)**.** Data structures with non-deterministic iteration order **MUST NOT** influence serialised output. All iteration **MUST** occur over a deterministically sorted sequence.

Requirement 8.5 ($\mathbb{Q}_{16}$ Arithmetic Exclusivity)**.** All identity-bearing numerical computations (axis functions, kernel evaluations, convergence checks, fingerprints) **MUST** be performed in $\mathbb{Q}_{16}$ arithmetic (Axiom 4.1). Platform-dependent floating-point **MUST NOT** participate.

9 Run Descriptor (RD)

9.1 Normative JSON Schema

```
{
 "schema_version": "2.1.0", // REQUIRED
 "run_id": "<hex sha256>", // REQUIRED (computed)

 "unicode_policy": {// REQUIRED
  "normalization_form": "NFC", // REQUIRED
  "forbidden_ranges": [],// OPTIONAL (default: [])
  "confusable_policy": null, // OPTIONAL
  "confusable_map": null // OPTIONAL
 },

 "canon_policy": {// REQUIRED
  "id_scheme": "content_hash", // REQUIRED
  "node_order": "lexicographic_utf8", // REQUIRED
  "edge_order": "lexicographic_utf8" // REQUIRED
 },

 "seed_policy": {// REQUIRED
  "mode": "fixed", // REQUIRED
  "seed": "0" // REQUIRED if mode=fixed
 },

 "embedding_policy": {// REQUIRED
  // Kernel bandwidth parameters (Q16 values)
  "sigma": [65536, 65536, 65536, 65536, 65536],
  // Spectral topology size n (domain parameter)
  "spectral_n": 13,
  // Axis weight vectors (arrays of Q16 values)
  "axis_weights": [[],[],[],[],[]]
 },

 "convergence_policy": {// REQUIRED
  // Minimum number of TIC observations before
  // crystallization is eligible
  "s_min": 5,
  // Stability window size for the condensing predicate
  "w": 3,
  // Contractivity threshold (Q16, < 65536 = 1.0)
  "rho_max": 64880
 },

 "registries": {// REQUIRED
  "operator": "<hex sha256>",
  "macro": "<hex sha256>",
  "obligation": "<hex sha256>",
  "observable": "<hex sha256>"
 },

 "toolchain": {// REQUIRED
  "name": "glyph",
```

```
    "version": "0.2.0",
    "platform": null // OPTIONAL
  },

  "digest_algorithm": "sha256" // OPTIONAL (default)
}
```

9.2 RD Computation Rules

Requirement 9.1 (RD Digest)**.** The `"run_id"` field **MUST** be computed by serialising the RD as canonical JSON with `"run_id"` set to `""`, then computing sha256 over the byte sequence, and encoding as lowercase hex.

Requirement 9.2 (RD Immutability)**.** Once a run has commenced, the RD **MUST NOT** be modified.

10 System Model

10.1 Core State

The foundry operates on a canonical state tuple:

$$\Sigma = (\mathcal{G},\ \mathbf{e},\ \mathcal{R},\ H,\ RD)$$

where $\mathcal{G}$ is the Information Graph (IR instance), $\mathbf{e} \in \mathbb{H}_5$ is the current embedding, $\mathcal{R} = (\mathcal{R}_{\text{op}}, \mathcal{R}_{\text{mac}}, \mathcal{R}_{\text{obl}}, \mathcal{R}_{\text{obs}})$ is the quadruple of active registries, H is the history (MEF chain + TIC), and *RD* is the Run Descriptor.

10.2 Execution Model: The Crystallization Pipeline

The execution model is an 8-stage deterministic pipeline. Each stage is implemented by an **Operator** with signature $\text{Op} : (\sigma, \theta) \to (\sigma', e)$ where σ is the pipeline state, θ is the configuration, σ' is the new state, and e is the operator evidence record. No stage **MUST NOT** execute before the preceding stage completes successfully.

Stage	Name	Input	Output
S0	Ingest	Raw source bytes	Normalised source + d_{src}
S1	Canon	Normalised source	Canonical IR graph $\mathcal{G}$ + d_{ir}
S2	Expand	$\mathcal{G}$ + $\mathcal{R}_{\text{mac}}$ (opt.)	Expanded IR + updated d_{ir}
S3	Sign	Canonical $\mathcal{G}$	$\mathbf{e} \in \mathbb{H}_5$ + ZFP($\mathbf{e}$) + genome_id
S4	Condense	Candidate + TIC	Crystal $\mathcal{C}$ (if convergent) or $\perp$
S5	Gate	$\mathcal{G}$ + $\mathbf{e}$ + $\mathcal{R}_{\text{obl}}$	Evidence + pass/fail
S6	Emit	Crystal + Evidence	Bundle directory + manifest
S7	Verify	Bundle + RD	Replay match / first divergence

Each stage appends an evidence record to the MEF chain (Section 16) and a trace record to H.

Theorem 10.1 (Pipeline Determinism)**.** *Given identical input and configuration θ, the pipeline produces bit-identical output:*

$$\mathcal{P}(\text{input}, \theta) = \mathcal{P}(\text{input}, \theta)$$

This follows from Axiom 4.1 (deterministic arithmetic), Definition 4.1 (deterministic hashing), and the requirement that all operators are pure functions of (σ, θ).

11 Unicode Policy

11.1 Normalisation

Requirement 11.1 (Normalisation Application)**.** Stage S0 (Ingest) **MUST** apply Unicode normalisation to raw source bytes using the form declared in `"unicode_policy.normalization_form"`. Default: NFC. A leading BOM (U+FEFF) **MUST** be stripped before normalisation.

Requirement 11.2 (Normalised Source Persistence)**.** The normalised source **MUST** be persisted as `source.norm.txt` in the bundle, encoded as UTF-8 without BOM.

11.2 Forbidden and Restricted Characters

Requirement 11.3 (Forbidden Range Rejection)**.** If `"forbidden_ranges"` is non-empty, the canonicaliser **MUST** scan the normalised source for any code point within a forbidden range. On first match, abort with an evidence record containing type, codepoint, byte offset, line, and column.

11.3 Confusable Policy

Requirement 11.4 (Confusable Detection)**.** If `"confusable_policy"` is `"warn"`, emit evidence for confusable pairs but do not abort. If `"reject"`, abort on the first confusable pair. Evidence records **MUST** contain: identifier pair, skeleton, and source locations.

11.4 Unicode-Specific Threats

(U1) **Composed vs. decomposed Devanagari.** The system **MUST** normalise to the declared form before any processing.

(U2) **Invisible characters.** Zero-width joiners (U+200D), non-joiners (U+200C), and directional markers (U+200E/F) **SHOULD** be in the default forbidden ranges unless explicitly permitted.

(U3) **Confusable attacks.** Visually similar identifiers from different scripts may bind to different IR nodes. The confusable policy mitigates this when enabled.

12 Canonical IR (Wire Format)

12.1 Single Truth Requirement

The foundry **MUST** define exactly one canonical IR wire format for transport and hashing. All surface languages **MUST** lower into this IR. Surface languages **MUST NOT** introduce parallel IRs.

12.2 IR Node Schema v1.0

Every node in the IR graph **MUST** conform to the following shape:

```
{
  "id": "<string>", // REQUIRED. Stable node identifier.
  "kind": "<NodeKind>", // REQUIRED.
  "name": "<string>", // REQUIRED.
  "params": ["<string>"], // OPTIONAL (Function nodes).
  "value": <str|int|bool>, // OPTIONAL (Literal nodes).
  "literal_type": "<string>", // OPTIONAL.
  "op": "<string>", // OPTIONAL (BinaryOp nodes).
  "callee": "<string>", // OPTIONAL (Call nodes).
  "properties": {}// OPTIONAL. Keys: strings, vals: str|int.
}
```

Node Kinds (Normative): `Module`, `Function`, `Block`, `If`, `Return`, `Call`, `Assignment`, `Literal`, `BinaryOp`, `Identifier`.

12.3 IR Edge Schema v1.0

```
{
  "src": "<string>", // REQUIRED.
  "dst": "<string>", // REQUIRED.
  "kind": "<string>", // REQUIRED.
  "ordinal": <integer> // OPTIONAL (default: 0).
}
```

Edge Kinds: `contains`, `next`, `condition`, `then_branch`, `else_branch`, `argument`, `target`, `value`, `left`, `right`, `callee_ref`.

12.4 IR Document Schema v1.0

```
{
  "schema_version": "1.0.0", // REQUIRED.
  "nodes": [<Node>, ... ],// REQUIRED. Sorted by prec_V.
  "edges": [<Edge>, ... ],// REQUIRED. Sorted by prec_E.
  "source_language": "<string>", // REQUIRED.
  "source_digest": "<hex sha256>", // REQUIRED.
  "embedding": {// REQUIRED (added in v2.1).
    "axes": [<Q16>, <Q16>, <Q16>, <Q16>, <Q16>],
    "fingerprint": "<hex sha256>",
    "genome_id": "<hex sha256>"
```

```
  }
}
```

12.5 Canonical Serialisation Invariants

Invariant 12.1 (Canonical JSON Form)**.** *The canonical serialisation **MUST** satisfy simultaneously:*

(i) Object keys sorted lexicographically by UTF-8 byte sequences.

(ii) `"nodes"` array sorted by $\prec_V$ (Section 13).

(iii) `"edges"` array sorted by $\prec_E$.

(iv) No floating-point literals; all numeric values are JSON integers.

(v) No whitespace between JSON tokens.

(vi) Minimal escape sequences: only `\"`, `\\`, `\n`, `\r`, `\t`, and `\uXXXX` for control characters below U+0020.

Invariant 12.2 (Hash Stability)**.** *For structurally identical IR graphs G_1, G_2 (isomorphic under type-preserving, label-preserving bijection), their canonical serialisations **MUST** be byte-identical, hence* $\mathrm{dig}(G_1) = \mathrm{dig}(G_2)$.

13 Deterministic Ordering Rules

13.1 Canonical Node Ordering ($\prec_V$)

Definition 13.1 (Node Sort Key)**.** $k_V(v) = \big(\mathrm{kind}(v),\ \mathrm{name}(v),\ \mathrm{id}(v)\big)$. Comparison: lexicographic on UTF-8 byte sequences of each component.

13.2 Canonical Edge Ordering ($\prec_E$)

Definition 13.2 (Edge Sort Key)**.** $k_E(e) = \big(\mathrm{kind}(e),\ \mathrm{src}(e),\ \mathrm{dst}(e),\ \mathrm{ordinal}(e)\big)$. String components compared lexicographically on UTF-8; ordinal compared numerically.

13.3 Stable Node ID Allocation

Requirement 13.1 (Content-Hash IDs)**.** When `"id_scheme"` is `"content_hash"`, each node ID **MUST** be:

$$\mathrm{id}(v) = \texttt{"n_"} \| \mathrm{sha256}\big(\mathrm{kind}(v) \| \texttt{":"} \| \mathrm{name}(v) \| \texttt{":"} \| \mathrm{scope_path}(v)\big)[0{:}12]$$

Requirement 13.2 (Collision Resolution)**.** Duplicate IDs receive monotonic suffixes `_dup1`, `_dup2`, etc., in source order.

14 Convergence Tracking

14.1 Temporal Information Certificate (TIC)

Definition 14.1 (TIC)**.** A **Temporal Information Certificate** for a genome g is a sequence of observations:

$$\text{TIC}_g = \big[(t_1, \delta_1, \kappa_1),\ (t_2, \delta_2, \kappa_2),\ \ldots,\ (t_N, \delta_N, \kappa_N)\big]$$

where:

- $t_k \in \mathbb{N}$ is the observation tick.
- $\delta_k \in \mathbb{Q}_{16}$ is the kernel distance $\delta(\mathbf{e}_k, \mathbf{e}_{k-1})$ between consecutive embeddings (with $\delta_1 = 0$).
- $\kappa_k \in \mathbb{Q}_{16}$ is the confidence: $\kappa_k = \min(k/s_{\text{min}},\ 1)$ where s_{min} is the minimum support threshold from the RD.

14.2 Condensation Predicate

Definition 14.2 (Condensing)**.** A TIC is **condensing** over a window of size w if the kernel distances are monotonically non-increasing over the last w entries:

$$\text{condensing}(\text{TIC}_g, w) \iff \forall\, j \in [N - w + 1,\ N - 1] :\ \delta_{j+1} \leq \delta_j$$

This is a strict monotonicity check on $\mathbb{Q}_{16}$ integers. No floating-point epsilon is involved.

14.3 Crystallization Condition

Definition 14.3 (Crystallization Condition)**.** A genome g is ready for crystallization if and only if:

$$|\text{TIC}_g| \geq s_{\text{min}} \ \wedge\ \text{condensing}(\text{TIC}_g, w)$$

where s_{min} and w are from the RD's `"convergence_policy"`.

14.4 Contractivity and Convergence

Theorem 14.1 (Convergence under Contractivity)**.** *If the information evolution process is **contractive** — i.e. there exists $\rho < 1$ such that $\delta(\mathbf{e}_{k+1}, \mathbf{e}^*) \leq \rho \cdot \delta(\mathbf{e}_k, \mathbf{e}^*)$ for some fixpoint $\mathbf{e}^*$ — then:*

1. *The TIC sequence $\{\delta_k\}$ converges monotonically to 0.*
2. *The crystallization condition is satisfied in finite time.*
3. *The resulting Crystal embedding satisfies $\delta(\mathbf{e}_{\mathcal{C}}, \mathbf{e}^*) \leq \frac{\rho^N}{1-\rho} \cdot \delta(\mathbf{e}_1, \mathbf{e}^*)$.*

Proof. By the Banach fixed-point theorem, a contractive map on a complete metric space has a unique fixpoint, and iterates converge geometrically. Since $\mathbb{H}_5$ with the kernel distance δ is a pseudometric space (metrizable by identifying equivalent points), and $\mathbb{Q}_{16}$ is discrete (hence complete), the conditions of Banach's theorem are satisfied when $\rho < 1$. □

The framework does **not** require all processes to be contractive. The TIC monitors convergence empirically; for non-contractive processes, the TIC simply never satisfies the crystallization condition.

15 Crystal: The Immutable Knowledge Unit

Definition 15.1 (Crystal)**.** A **Crystal** $\mathcal{C}$ is an immutable, certified information container:

$$\mathcal{C} = (\mathrm{id}_{\mathcal{C}},\ \mathrm{core},\ s,\ L,\ \mathrm{certs},\ h_\theta,\ \mathcal{E})$$

where:

- $\mathrm{id}_{\mathcal{C}} = \mathrm{dig}(\mathrm{core})$ is the unique Crystal identifier.
- $\mathrm{core} = (\texttt{graph_id},\ \texttt{sig_id},\ \texttt{genome_id})$ references the Information Graph, its spectral signature, and its genome.
- $s \in \mathbb{N}$ is the support count (number of TIC observations).
- L is the lineage (parent Crystals, merge reason, commit tick).
- certs is a set of Projection Certificates.
- $h_\theta = \mathrm{dig}(\theta)$ is the configuration hash.
- $\mathcal{E}$ is a set of evidence references.

Axiom 15.1 (Crystal Authority)**.** A Crystal exists if and only if it was produced by the crystallization protocol (Protocol 15.1). No other construction is valid. A Crystal is never modified after creation.

Protocol 15.1 (Crystallization)**.** Given a candidate c with embedding $\mathbf{e}_c$:

1. Append $(\mathrm{tick},\ \delta_k,\ \kappa_k)$ to $\mathrm{TIC}_{\mathrm{genome}(c)}$.
2. If the crystallization condition (Definition 14.3) is satisfied:
   (a) Construct the Crystal $\mathcal{C}$ from c.
   (b) Append a block to the MEF chain (Section 16).
   (c) Return $\mathcal{C}$.
3. Otherwise: return $\perp$ (candidate remains ephemeral).

15.1 Crystal JSON Schema

```
{
  "schema_version": "2.1.0",
  "crystal_id": "<hex sha256>",
  "core": {
    "graph_id": "<hex sha256>",
    "signature_id": "<hex sha256>",
    "genome_id": "<hex sha256>"
  },
  "embedding": {
    "axes": [<Q16>, <Q16>, <Q16>, <Q16>, <Q16>],
    "fingerprint": "<hex sha256>"
  },
```

```
  "support_count": <integer>,
  "tic_final": {
    "delta_last": <Q16>,
    "kappa_last": <Q16>,
    "condensing_window": <integer>
  },
  "lineage": {
    "parent_ids": ["<hex sha256>"],
    "merge_reason": "<string>",
    "commit_tick": <integer>
  },
  "config_hash": "<hex sha256>",
  "evidence_refs": ["<hex sha256>"],
  "created_at": "<ISO8601>"
}
```

16 Morphogenetic Evidence Framework (MEF)

Definition 16.1 (Evidence Chain)**.** The **Morphogenetic Evidence Framework** is an append-only hash chain:

$$\text{MEF} = [B_0, B_1, \ldots, B_N]$$

where each block $B_k = (h_k, h_{k-1}, \tau_k, t_k)$ contains:

- $h_k = \text{SHA256}(h_{k-1} \| \text{JCS}(\text{state}_k))$: the block hash, chained to the previous block.
- h_{k-1}: the hash of the preceding block ($h_{-1} =$ `0x00…0` for the genesis block).
- $\tau_k \in$ String: the transition type (e.g. `"ingest"`, `"canon"`, `"sign"`, `"condense"`, `"certify"`, `"commit"`).
- $t_k \in \mathbb{N}$: the tick at which the block was created.

Theorem 16.1 (Chain Integrity)**.** *The MEF chain satisfies:*

1. ***Tamper detection:*** *modification of any block B_j invalidates all subsequent hashes $h_{j+1}, \ldots, h_N$.*
2. ***Ordering:*** *the chain imposes a total order on events.*
3. ***Verifiability:*** verify_chain(MEF) *recomputes all hashes in $O(N)$ and detects any inconsistency.*

Definition 16.2 (Operator Evidence)**.** Each transformation step in the pipeline produces an evidence record:

$$e_k = (\text{step},\ \text{role},\ \text{operator},\ \text{dig}(\sigma_{\text{before}}),\ \text{dig}(\sigma_{\text{after}}),\ \text{dig}(\text{evidence}_k))$$

Definition 16.3 (Replay Verification)**.** A process is **replay-verified** if re-executing the pipeline with the same input and configuration produces bit-identical evidence digests:

$$\text{replay}(\text{input}, \theta) = \text{original}(\text{input}, \theta) \iff \forall k : e_k^{\text{replay}} = e_k^{\text{original}}$$

16.1 MEF Block JSON Schema

```
{
 "block_index": <integer>,
 "block_hash": "<hex sha256>",
 "prev_hash": "<hex sha256>",
 "transition_type": "<string>",
 "tick": <integer>,
 "state_digest": "<hex sha256>",
 "operator_evidence": {
  "step": <integer>,
  "role": "<string>",
  "operator": "<string>",
  "before_digest": "<hex sha256>",
  "after_digest": "<hex sha256>",
  "evidence_digest": "<hex sha256>"
 }
}
```

17 Gate System

Definition 17.1 (Constraint Lattice)**.** A **Constraint Lattice** on $\mathbb{H}_5$ is the intersection of half-spaces:

$$\mathcal{K} = \bigcap_i \{\mathbf{e} \in \mathbb{H}_5 \mid g_i(\mathbf{e}) \geq \theta_i\}$$

where each $g_i : \mathbb{H}_5 \to \mathbb{R}$ is a constraint function and θ_i is its threshold. The lattice is feasible iff $\mathcal{K} \neq \emptyset$.

Definition 17.2 (Proof-of-Resonance (PoR))**.** The PoR is a finite state machine with states $\{S, L, V, C\}$ (Search, Lock, Verify, Commit) and transition condition:

$$\text{FIRE} \iff \text{valid} \ \wedge \ |\Delta\Pi| \leq \varepsilon \ \wedge \ \Phi \geq \phi^* \ \wedge \ \Delta V < 0 \ \wedge \ d \geq d_{\text{min}}$$

where $\Delta\Pi$ is path invariance, Φ is coherence, ΔV is volume change, and d is diversity. These conditions are defined on $\mathbb{H}_5$ metrics, not on domain-specific properties.

17.1 Obligation Registry Schema

```
{
 "schema_version": "1.0.0",
 "type": "obligation_registry",
 "entries": [
  {
   "id": "<string>", // REQUIRED.
   "version": "<semver>", // REQUIRED.
   "class": "<hard|soft>", // REQUIRED.
   "description": "<string>", // REQUIRED.
   "predicate": "<string>", // REQUIRED.
   "threshold": <Q16>, // OPTIONAL (soft: default 32768).
```

```
    "constraint_function": "<string>", // OPTIONAL (ref to g_i).
    "axis_index": <integer> // OPTIONAL (which H5 axis).
  }
 ]
}
```

Requirement 17.1 (Hard Obligation Failure)**.** If any hard obligation evaluates to "fail" during Gate (S5), the run **MUST** abort with exit code `2`. Partial bundles **MUST NOT** be emitted.

18 Contractive Fixpoint Iteration

Definition 18.1 (Fixpoint Operator Stack)**.** A sequence of p operators $T_1, \ldots, T_p$, each $T_j : \mathbb{H}_5 \to \mathbb{H}_5$, composed as:

$$T = T_p \circ \cdots \circ T_1$$

If T has spectral radius $\rho(T) < 1$, then the iteration $\mathbf{e}_{k+1} = T(\mathbf{e}_k)$ converges to a unique fixpoint $\mathbf{e}^* = T(\mathbf{e}^*)$.

Theorem 18.1 (Contractivity Verification)**.** *For a linear operator $T = W\mathbf{e} + \mathbf{b}$ with matrix $W \in \mathbb{R}^{5\times 5}$, contractivity is verified by power iteration:*

$$\rho(W) = \lim_{k\to\infty} \|W^k v\|^{1/k} < 1$$

for a random initial vector v. In practice, 30 iterations suffice to determine ρ to 6 significant digits.

19 Registries

Four registries **MUST** exist as versioned, content-addressed JSON objects:

19.1 Operator Registry

```
{
 "schema_version": "1.0.0",
 "type": "operator_registry",
 "entries": [
  {
   "id": "<string>", "version": "<semver>",
   "input_types": ["<NodeKind>"],
   "output_type": "<NodeKind>",
   "constraints": ["<string>"],
   "evidence_schema": {}
  }
 ]
}
```

19.2 Macro Registry

```
{
 "schema_version": "1.0.0",
 "type": "macro_registry",
 "entries": [
   {
    "pattern": "<string>",
    "precedence": <integer>,
    "expansion": {"nodes": [],"edges": []},
    "bindings": {}
   }
 ]
}
```

19.3 Observable Registry

```
{
 "schema_version": "1.0.0",
 "type": "observable_registry",
 "entries": [
   {
    "id": "<string>",
    "version": "<semver>",
    "arity": <integer>,
    "deterministic": true,
    "computation_ref": "<string>"
   }
 ]
}
```

19.4 Registry Hash Binding

Requirement 19.1 (Registry Digest in RD)**.** All registries **MUST** be referenced in the RD by their `SHA-256` digest.

Requirement 19.2 (Registry Immutability)**.** Registries **MUST NOT** be modified during a run.

20 Macro Expansion

Requirement 20.1 (Deterministic Expansion)**.** Macro expansion **MUST** be deterministic given (normalised source, RD, registries).

Requirement 20.2 (Precedence Resolution)**.** When multiple patterns match, the entry with the lowest precedence value wins. If tied, the lexicographically smaller pattern wins. If both identical, abort with conflict evidence.

Requirement 20.3 (Post-Expansion Re-Canonicalisation)**.** After all expansions, re-canonicalise (recompute node IDs, restore ordering) before proceeding.

21 Reference Surface Language: Sanskroot-Devanagari

21.1 Minimum Vocabulary (Normative)

Surface Token	Transliteration	IR Node Kind
कार्य	kārya	`Function`
यदि	yadi	`If`
अन्यथा	anyathā	else branch of `If`
दर्शय	darśaya	`Call` (callee = `print`)
निवृत्ति	nivṛtti	`Return`
सत्य	satya	`Literal` (boolean, true)
असत्य	asatya	`Literal` (boolean, false)
मान	māna	`Assignment`

Requirement 21.1 (Devanagari Identifiers)**.** Identifiers **MUST** permit Devanagari letters (U+0900–U+097F), combining marks, and ASCII digits.

22 Bundle and Evidence Format

22.1 Bundle Layout (Normative)

Path	Description
`source.norm.txt`	Normalised source.
`ir.json`	Canonical IR document (incl. embedding).
`rd.json`	Run Descriptor.
`manifest.json`	All digests (Section 22.2).
`trace.json`	Per-stage trace records.
`mef.json`	MEF hash chain (Section 16).
`crystal.json`	Crystal record (if crystallized; else absent).
`tic.json`	TIC observation sequence.
`evidence.json`	Gate/obligation evidence.
`artifacts/`	Emitted outputs.

22.2 Manifest Schema

```
{
  "schema_version": "2.1.0",
  "created_at": "<ISO8601>",
  "rd_digest": "<hex sha256>",
  "source_digest": "<hex sha256>",
  "ir_digest": "<hex sha256>",
  "mef_digest": "<hex sha256>",
  "evidence_digest": "<hex sha256>",
  "trace_digest": "<hex sha256>",
  "tic_digest": "<hex sha256>",
  "crystal_digest": "<hex sha256 | null>",
  "embedding_fingerprint": "<hex sha256>",
```

```
  "genome_id": "<hex sha256>",
  "registry_digests": {
    "operator": "<hex sha256>",
    "macro": "<hex sha256>",
    "obligation": "<hex sha256>",
    "observable": "<hex sha256>"
  },
  "artifact_digests": {"<path>": "<hex sha256>" },
  "ir_schema_version": "1.0.0",
  "toolchain": {"name": "<string>", "version": "<semver>" }
}
```

22.3 Verification

Requirement 22.1 (First Divergence Reporting)**.** On any digest mismatch, the verifier **MUST** report: file path, manifest field, expected digest, actual digest. The verifier **SHOULD** also report the byte offset of the first differing byte.

23 CLI Interface

23.1 Binary Name

The CLI binary **MUST** be named `glyph`.

23.2 Commands and Exit Codes

23.2.1 `glyph init-rd`

Emit a default RD. Exit: `0` success, `1` I/O error.

23.2.2 `glyph run`

Execute the full 8-stage pipeline and emit a bundle.

Flag	Type	Description
`--input`, `-i`	path	Source file. **REQUIRED**.
`--rd`	path	RD file. **REQUIRED**.
`--output-dir`, `-o`	path	Bundle dir. Default: `./bundle/`.
`--expand`	bool	Enable S2. Default: true.
`--crystallize`	bool	Enable S4 convergence check. Default: true.
`--frontend`	string	Frontend ID. Default: `sanskroot`.

Exit codes:

0	Success; bundle emitted (Crystal present if converged).
1	I/O or configuration error.
2	Hard obligation failure.
3	Parse or normalisation error.
4	Forbidden character detected.
5	Confusable policy rejection.
6	Macro conflict.
7	Registry digest mismatch.
8	MEF chain integrity failure.
9	Contractivity bound exceeded ($\rho \geq 1$).

Atomicity: On non-zero exit, no partial bundle directory **MUST** remain.

23.2.3 `glyph verify`

Verify bundle integrity with optional full replay.

Exit: `0` all match, `10` digest mismatch, `11` replay mismatch, `12` missing file, `13` MEF chain tamper detected.

23.2.4 `glyph dump-ir`

Parse and canonicalise, emit IR to stdout. Exit: `0`, `1`, `3`.

23.2.5 `glyph embed`

Compute and print the $\mathbb{H}_5$ embedding and fingerprint for a source file.

Exit: `0` success, `1` I/O error, `3` parse error.

Output format: five $\mathbb{Q}_{16}$ values (tab-separated) followed by a newline, then the fingerprint hash.

24 Golden Test Vectors

A conforming implementation **MUST** produce byte-identical canonical IR for each test vector when using the default RD.

24.1 Test Vector 1: Minimal Print

Source (`tv1.sk`):

```
(*@\deva{कर्य}@*) (*@\deva{मुख्य}@*)() {
   (*@\deva{दर्शय}@*)("(*@\deva{नमस्ते}@*)");
}
```

Expected IR shape: Module → Function(मुख्य) → Block → Call(print) → Literal(नमस्ते, string).

Expected embedding: Five $\mathbb{Q}_{16}$ values computed via the default axis functions. The embedding **MUST** be reproducible across runs.

24.2 Test Vector 2: Conditional Branch

Source (`tv2.sk`):

```
(*@\deva{कार्य}@*) (*@\deva{जैसा}@*)(x) {
   (*@\deva{यदि}@*) (x > 0) {
      (*@\deva{दर्शाय}@*)("(*@\deva{धनात्मक}@*)");
   }(*@\deva{अन्यथा}@*) {
      (*@\deva{दर्शाय}@*)("(*@\deva{अधनात्मक}@*)");
   }
}
```

Expected IR shape: Module → Function → Block → If (with BinaryOp(gt), then-Block, else-Block).

24.3 Test Vector 3: Multi-Function with Return

Source (`tv3.sk`):

```
(*@\deva{कार्य}@*) (*@\deva{योग}@*)(a, b) {
   (*@\deva{निवृत्ति}@*) a + b;
}
(*@\deva{कार्य}@*) (*@\deva{मुख्य}@*)() {
   (*@\deva{मान}@*) (*@\deva{परिणाम}@*) = (*@\deva{योग}@*)(3, 4);
   (*@\deva{दर्शाय}@*)((*@\deva{परिणाम}@*));
}
```

24.4 Golden Path Determinism Test

Requirement 24.1 (Golden Path)**.** For each test vector, the implementation **MUST**: (i) execute `glyph run` twice with the default RD, (ii) assert byte-identical `ir.json`, (iii) assert identical manifest digests, (iv) assert identical $\mathbb{H}_5$ embeddings and fingerprints, (v) assert identical TIC records if crystallization was attempted.

25 Domain Adaptation and Integration Boundary

25.1 Domain Adapter

Definition 25.1 (Domain Adapter)**.** A **Domain Adapter** $\mathcal{D}$ is a tuple:

$$\mathcal{D} = (\mathcal{T}_V,\, \mathcal{T}_E,\, \mathcal{O}_1 \ldots \mathcal{O}_m,\, f_1 \ldots f_5,\, \pi,\, \varepsilon,\, n)$$

where:

- $\mathcal{T}_V, \mathcal{T}_E$: vertex and edge type alphabets.
- $\mathcal{O}_1, \ldots, \mathcal{O}_m$: the observable set.
- $f_1, \ldots, f_5$: the five axis functions for $\mathbb{H}_5$.
- π : Entity $\to \mathcal{G}$: the **projection function** (domain object → Information Graph).

- $\varepsilon : \mathcal{G} \to$ Entity: the **emission function** (Information Graph $\to$ domain object).
- n: the spectral topology size.

Definition 25.2 (Roundtrip Certification)**.** A Crystal $\mathcal{C}$ is **roundtrip-certified** under adapter $\mathcal{D}$ if:

$$\delta\big(\mathbf{e}(\pi(\varepsilon(\mathcal{G}_\mathcal{C}))),\ \mathbf{e}(\mathcal{G}_\mathcal{C})\big) \leq \delta_{\text{max}}$$

i.e. emitting the Crystal's graph back into domain format and re-projecting yields an embedding within δ_{max} of the original.

25.2 Example Domain Adapters

Domain	$\mathcal{T}_V$	$\mathcal{T}_E$	n	**Axes**
Software	Module, Func, Type, …(14)	Contains, Calls, …(18)	13	Coupling, Density, Complexity, Symmetry, Entropy
Chemistry	Atom, Bond, Group, Ring	Single, Double, Aromatic, Ionic	18	Rigidity, Reactivity, Solubility, Symmetry, Accessibility
Law	Clause, Definition, Condition	Defines, Requires, Contradicts	7	Interconnection, Specificity, Complexity, Consistency, Coverage
Music	Note, Chord, Phrase, Section	Follows, Harmonizes, Resolves	12	Harmony, Rhythm, Texture, Tension, Resolution

25.3 Integration Boundary

The Glyph Foundry is standalone. Integration with external systems is achieved exclusively through DTO adapters.

Requirement 25.1 (Version Negotiation)**.** An adapter **MUST** declare the Glyph IR schema versions it supports. Unsupported versions **MUST** be rejected with an explicit error.

Requirement 25.2 (Upgrade/Downgrade)**.** Lossy conversions between schema versions **MUST** be explicitly flagged and **MUST NOT** be performed silently.

26 Composition: Recursive Crystallization

Definition 26.1 (HyperCrystal)**.** A **HyperCrystal** is a Crystal whose Information Graph contains references to other Crystals as substructures:

$$\mathcal{H} = \big(\mathcal{C}_{\text{root}},\ \{\mathcal{C}_1, \ldots, \mathcal{C}_k\},\ E_{\text{compose}}\big)$$

The embedding of a HyperCrystal is the weighted mean of its constituent embeddings:

$$\mathbf{e}_{\mathcal{H}} = \frac{1}{k+1}\left(\mathbf{e}_{\text{root}} + \sum_{i=1}^{k} \mathbf{e}_i\right)$$

A HyperCrystal is itself subject to TIC convergence tracking and crystallization, enabling **recursive composition**.

Proposition 26.1 (Closure under Composition)**.** *The set of Crystals is closed under HyperCrystal composition: if $\mathcal{C}_1, \ldots, \mathcal{C}_k$ are valid Crystals, then $\mathcal{H}(\mathcal{C}_1, \ldots, \mathcal{C}_k)$ is a valid Crystal (assuming TIC convergence).*

27 Formal Properties

The following eight properties hold for any valid Domain Adapter, independent of domain:

P1 Determinism. Identical input + identical configuration = bit-identical output. (Axiom 4.1, Theorem 10.1)

P2 Convergence. Under contractivity ($\rho < 1$), crystallization occurs in finite time. (Theorem 14.1)

P3 Tamper Evidence. Any modification to the MEF chain is detectable. (Theorem 16.1)

P4 Replay Verifiability. Any crystallization process can be independently re-executed and verified. (Definition 16.3)

P5 Domain Agnosticism. The core algebraic structure ($\mathbb{H}_5$, TIC, MEF, PoR, K_n) is independent of domain semantics. (Definition 25.1)

P6 Immutability. Crystals are never modified after creation. (Axiom 15.1)

P7 Composability. Crystals compose recursively into HyperCrystals, preserving all guarantees. (Proposition 26.1)

P8 Certifiability. Roundtrip certification provides a quantitative measure of representation fidelity. (Definition 25.2)

28 Threat and Failure Model

Code	Label	Description
FC-01	ParseError	Syntax error, unsupported language, malformed RD.
FC-02	NormError	Unicode normalisation failure.
FC-03	ForbiddenChar	Forbidden code point in normalised source.
FC-04	ConfusableReject	Confusable policy rejection.
FC-05	MacroConflict	Ambiguous macro expansion.
FC-06	ObligationFail	Hard obligation failure.
FC-07	RegistryMismatch	Registry digest mismatch between RD and manifest.
FC-08	DigestMismatch	File digest does not match manifest.
FC-09	ReplayMismatch	Full replay produces different manifest digest.
FC-10	NodeIDCollision	Content-hash collision (warning).
FC-11	ToolchainMismatch	Toolchain version differs (warning).
FC-12	MEFTamper	MEF chain integrity failure.
FC-13	ContractivityBreach	Spectral radius $\rho \geq 1$ detected during fixpoint iteration.
FC-14	ConvergenceStall	TIC fails to condense after configurable maximum ticks.
FC-15	ArithOverflow	$\mathbb{Q}_{16}$ arithmetic overflow in axis function or kernel computation.

Requirement 28.1 (Atomic Bundle Emission)**.** On any failure during S0–S6, the output directory **MUST NOT** contain a partial bundle.

Requirement 28.2 (Digest Algorithm Pinning)**.** The digest algorithm **MUST** be `SHA-256` unless the RD explicitly declares SHA-384 or SHA-512.

29 Repository Blueprint

Crate	Responsibility
`glyph-q16`	$\mathbb{Q}_{16}$ fixed-point arithmetic library.
`glyph-rd`	RD parsing, validation, digest.
`glyph-canon`	Canonicalisation, ordering, ID allocation.
`glyph-ir`	IR schema types, serialisation, validation.
`glyph-registry`	Registry types (operator, macro, observable, obligation).
`glyph-embed`	$\mathbb{H}_5$ embedding, axis functions, kernel, fingerprint.
`glyph-tic`	TIC tracking, condensation, crystallization protocol.
`glyph-crystal`	Crystal construction, HyperCrystal composition.
`glyph-mef`	MEF chain construction and verification.
`glyph-gate`	Constraint lattice, obligation evaluation, PoR.
`glyph-frontends`	Sanskroot-Devanagari + optional frontends.
`glyph-expand`	Macro expansion engine.
`glyph-bundle`	Bundle assembly, manifest, atomic emission.
`glyph-verify`	Digest verification, replay, divergence reporting.
`glyph-cli`	CLI binary, argument parsing, exit code mapping.

30 Evolution Staircase

30.1 Stable Core (MUST remain invariant)

(i) $\mathbb{Q}_{16}$ deterministic arithmetic.

(ii) The determinism contract.

(iii) Single canonical IR wire format.

(iv) $\mathbb{H}_5$ embedding space definition and fingerprint.

(v) TIC convergence tracking and crystallization protocol.

(vi) MEF chain structure.

(vii) Bundle layout and manifest digest rules.

(viii) Registry hash binding.

(ix) CLI exit code semantics.

(x) Canonical JSON serialisation rules.

30.2 Mutable Surface (MAY evolve)

(i) New glyph operator sets and macro expansions.

(ii) New obligation/profile families.

(iii) Additional IR node/edge kinds (under major schema version bump).

(iv) New axis functions and observable definitions.

(v) Additional CLI commands or flags.

(vi) New Domain Adapters.

31 Definition of Done (Production Readiness)

An implementation is production-ready if and only if:

1. The reference Sanskroot-Devanagari frontend lowers all three golden test vectors to canonical IR deterministically.
2. $\mathbb{H}_5$ embeddings are computed correctly in $\mathbb{Q}_{16}$ arithmetic and reproduce across runs.
3. TIC convergence tracking correctly detects condensation and triggers crystallization.
4. Crystal records are constructed with correct digests and lineage.
5. MEF chain is constructed, append-only, and verifiable.
6. Bundle emission produces the normative layout and a valid manifest.
7. Verification reproduces all digests and detects divergence with first-divergence reporting.
8. Full replay produces byte-identical bundles.
9. Registries (operator, macro, obligation, observable) are versioned and digest-bound.
10. CI contains the golden path determinism test for all test vectors.
11. All exit codes are correctly emitted.
12. Atomic bundle emission is enforced.
13. A conformance statement listing each **MUST/MUST NOT** requirement exists.

A Canonical JSON Serialisation Algorithm

1. **Null:** `null`.
2. **Boolean:** `true` or `false`.
3. **Integer:** decimal, no leading zeros, negative with leading –.

4. **String:** `"`, then per character: `\"`, `\\`, `\n`, `\r`, `\t` for the respective controls; `\uXXXX` for other U+0000–U+001F; all else as UTF-8. Close with `"`.
5. **Array:** `[`, elements separated by `,`, then `]`.
6. **Object:** sort keys lexicographically by UTF-8 bytes. Emit `{`, key-value pairs separated by `,`, then `}`.
7. **Floating-point:** forbidden. **MUST** reject.

B Digest Computation Reference

All digests: read input as raw bytes (for JSON: canonical serialisation), compute SHA-256, encode as 64-character lowercase hex. The `"created_at"` field in manifests is excluded from all digest computations.